PROJECT DOCUMENTATION

CONTACT MANAGER

Project Title: CONTACT MANAGER

Team ID: SWTID-2026-1594

Team Leader: KAVIYA.S

Email: kaviya26shanmugam2006@gmail.com

Team Members:

RESHMA.D – reshmadevaraj2006@gmail.com

TAMILVANI.N – tamilvani0107@gmail.com

BALA DHARSHINI.P – dharshini4446@gmail.com

1. Introduction

The Contact Manager project is designed to help users store and manage contact information digitally. Instead of maintaining phone numbers and email addresses in notebooks or scattered files, this application provides a centralized system where users can keep all contact details in one place.

The system allows users to add, edit, delete, and search contact information easily. It improves accessibility and organization of data. The main aim of the project is to demonstrate how a simple software application can be used to manage personal or professional contacts efficiently.

2. Project Overview

The Contact Manager application provides a structured way to manage contact details. The system maintains a list of contacts where each entry includes name, phone number, and email address. Users can update the information whenever required.

This project demonstrates the implementation of basic CRUD operations (Create, Read, Update, Delete). These operations form the core functionality of many modern applications. By implementing these features, the project shows how data can be stored, retrieved, and modified in a simple system.

3. Architecture

The Contact Manager system follows a simple architecture consisting of a user interface, application logic, and database. The user interacts with the interface to perform actions such as adding or viewing contacts.

The application logic processes the user requests and communicates with the database to store or retrieve data. This layered structure makes the application organized and easier to maintain.

4. Tools and Technologies

The development of the Contact Manager project involves several tools and technologies that help in building and managing the application.

• Frontend: React.js for creating interactive user interfaces.

• Backend: Node.js and Express.js for handling server operations.

• Database: MongoDB to store and manage contact records.

• Development Tool: Visual Studio Code.

• Version Control: Git for tracking project changes.

5. Folder Structure

A well■organized folder structure helps developers maintain the project efficiently. The project usually contains separate folders for frontend and backend components.

Contact-Manager/

|-- client (frontend files)

|-- server (backend files)

|-- models (database models)

|-- routes (API routes)

6. Running the Application

The application can be executed locally by installing the required dependencies and starting the server. This allows the user to test the features of the Contact Manager system.

Steps:

1. Install required packages using npm install.

2. Start the backend server.

3. Run the frontend application.

4. Open the browser and access the application interface.

7. API Endpoints

The backend system provides several API endpoints that allow the frontend application to interact with the database.

POST /contacts – Add a new contact.

GET /contacts – Retrieve all contacts.

PUT /contacts/id – Update an existing contact.

DELETE /contacts/id – Remove a contact.

8. Authentication

Authentication is used to ensure that only authorized users can access the system. Secure login mechanisms help protect contact data from unauthorized access.

Technologies such as JSON Web Tokens (JWT) can be used to implement user authentication and secure communication between the client and server.

9. User Interface

The user interface of the Contact Manager application is designed to be simple and user■friendly. Users can easily navigate through different sections of the application.

Main pages include:

• Dashboard showing the contact list.

• Form page for adding or editing contacts.

• Search feature to quickly find contacts.

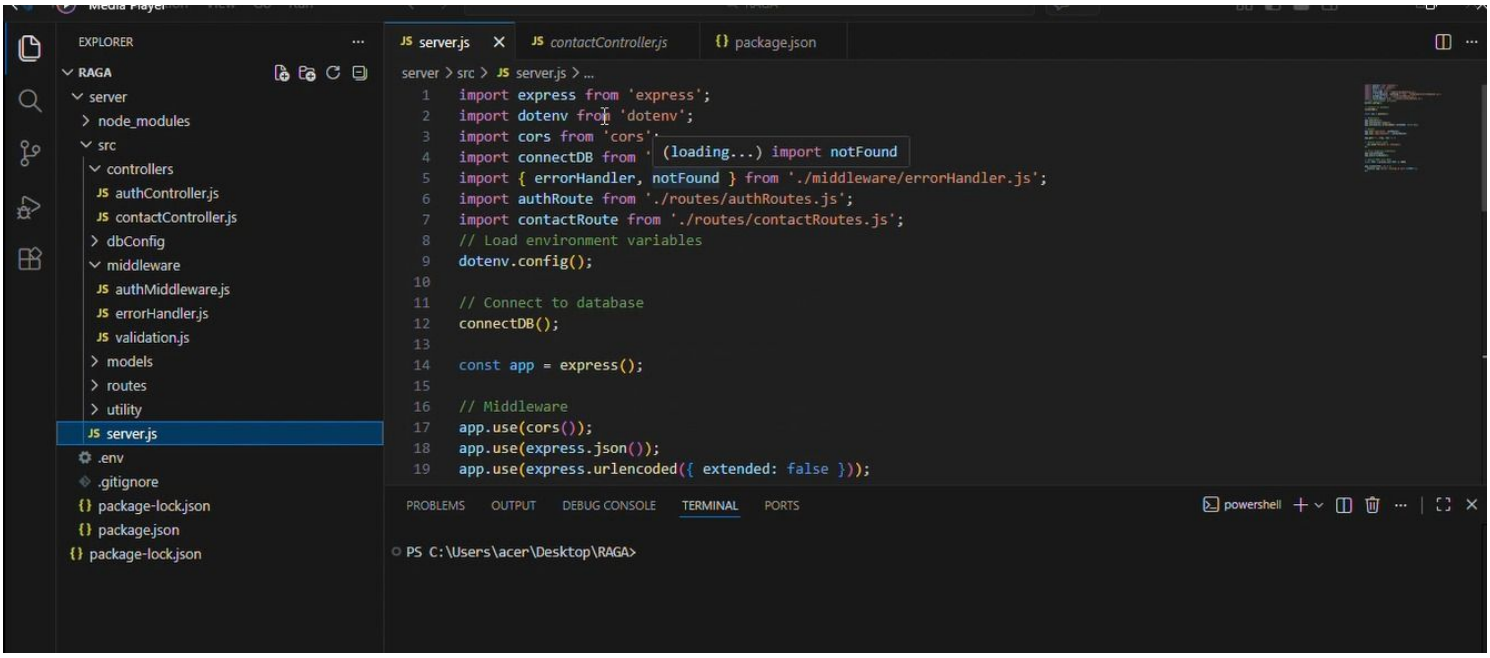
10. Testing

Testing ensures that the application works correctly and that all features perform as expected. Both manual testing and basic API testing can be performed during development.

Tools such as Postman and browser developer tools can help developers test API responses and identify issues in the system.

11. Screenshots or Demo

The following pages are reserved for screenshots of the Contact Manager application interface, demonstrating how the system works during execution.



Ragavi's Workspace

GET Get data | GET Untitled Request | GET Untitled Request | POST http://localhost:5000/

http://localhost:5000/api/auth/login

POST http://localhost:5000/api/auth/login Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL JSON

```
{
    "email":"tree@gmail.com",
    "password":"123456"
}
```

Body Cookies Headers 8 Test Results — 200 OK · 127 ms · 556 B

```
{
    "success": true,
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJ1c2VySWQiOiI2OWFlYjlhNTI2Y2U0NGNmMjM3YTQxMGEiLCJpYXQiOjE3NzMwNTg1MTksImV4cCI6MTc3NTY1MDUxOX0.
        34iFMqs_twwyCHTVj0E1KoBO4NmTShEGrlWhLZqq9Bo",
    "user": {
        "id": "69aeb9a526ce44cf237a410a",
        "name": "treeuser",
        "email": "tree@gmail.com"
    }
}
```

localhost:5000

Backend is running

Ragavi's Workspace

GET Get data | GET Untitled Request | GET Untitled Request | POST http://localhost:5000/

http://localhost:5000/api/auth/registe

POST http://localhost:5000/api/auth/registe Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL JSON

```
{
    "name":"treeuser",
    "email":"tree@gmail.com",
    "password":"123456"
}
```

Body Cookies Headers 8 Test Results — 201 Created · 172 ms · 561 B

```
{
    "success": true,
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJ1c2VySWQiOiI2OWFlYjlhNTI2Y2U0NGNmMjM3YTQxMGEiLCJpYXQiOjE3NzMwNTg0NjksImV4cCI6MTc3NTY1MDQ2OX0.
        uHqG4O2qMaOUkxKV0UikkG58UpofaueiNfzSAKAfthk",
    "user": {
        "id": "69aeb9a526ce44cf237a410a",
        "name": "treeuser",
        "email": "tree@gmail.com"
    }
}
```

12. Known Issues

In the current version, some advanced features such as cloud synchronization, backup, and contact grouping may not be fully implemented.

13. Future Enhancements

- Add profile pictures for contacts.
- Implement cloud backup and synchronization.
- Add contact groups and categories.
- Develop a mobile version of the application.